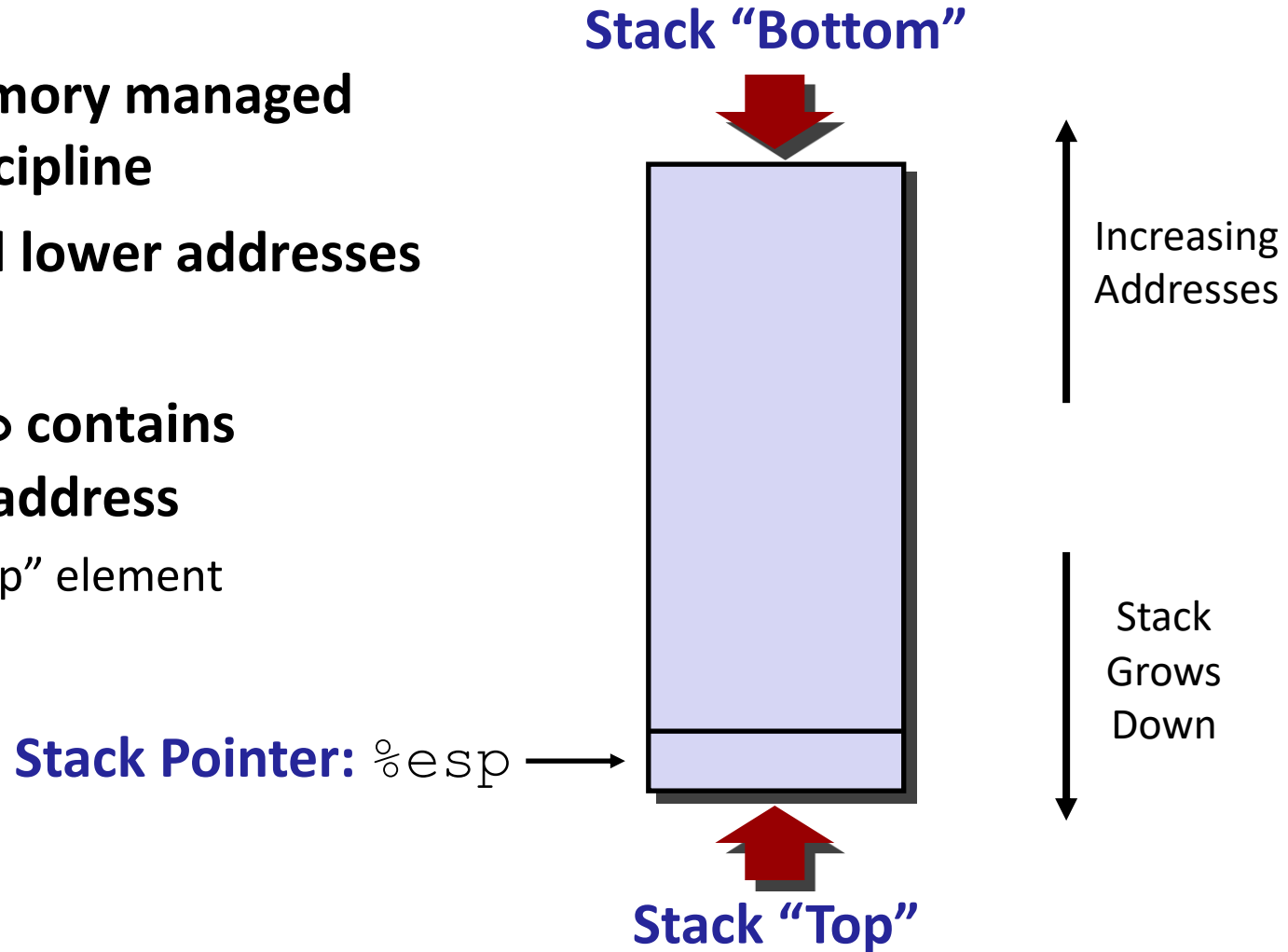


The Stack and Procedures

- **Stack: LIFO structure in memory managed by rsp.**
 - push rax: Decrement rsp, store value.
 - pop rbx: Load value, increment rsp.
- **Procedures:**
 - call function_name: Pushes the Return Address (RIP) to the stack.
 - ret: Pops the address from the stack and jumps to it.
- **Calling Convention (Linux x86-64):**
 - Arguments passed in: rdi, rsi, rdx, rcx, r8, r9

IA32 Stack

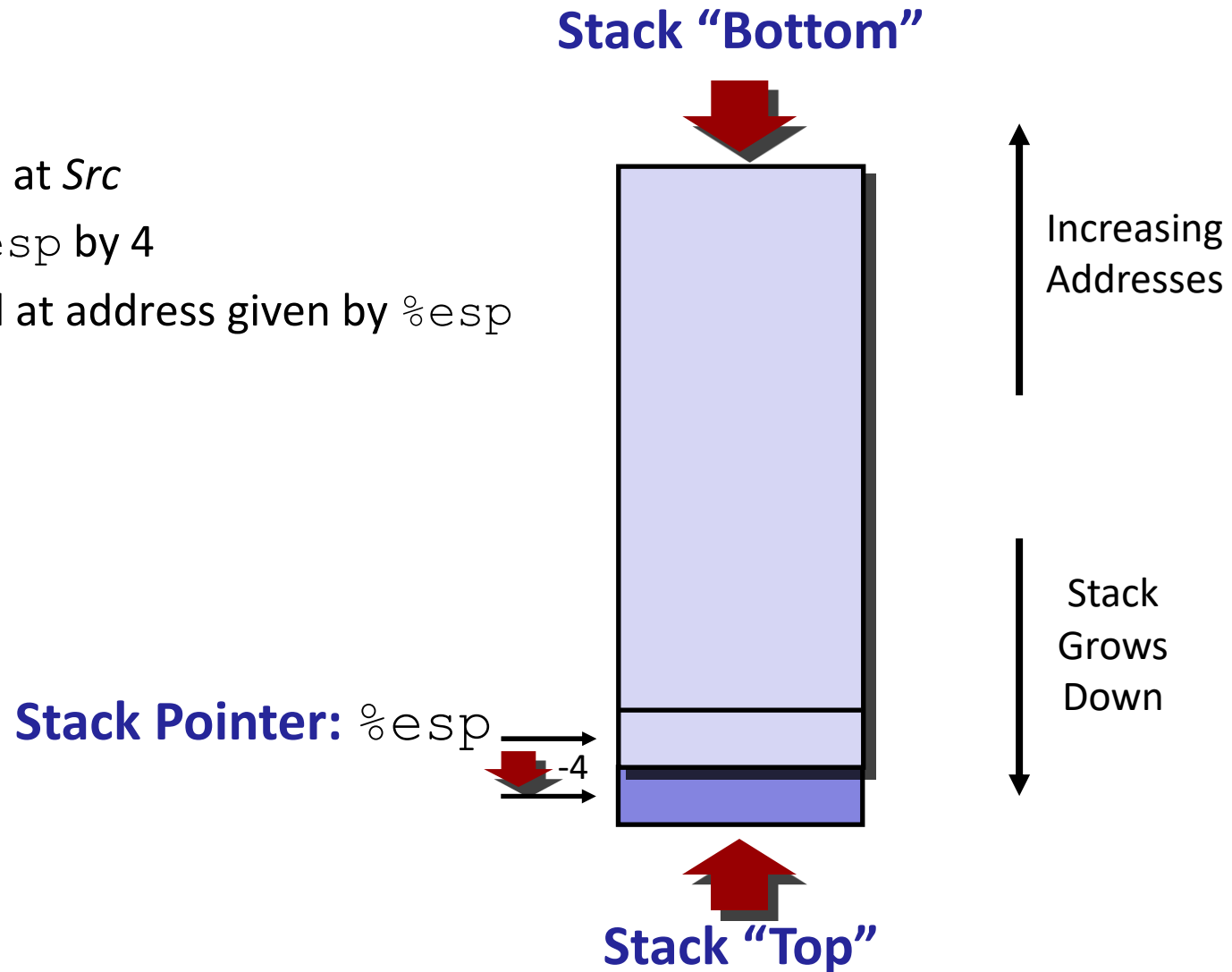
- Region of memory managed with stack discipline
- Grows toward lower addresses
- Register `%esp` contains lowest stack address
 - address of “top” element



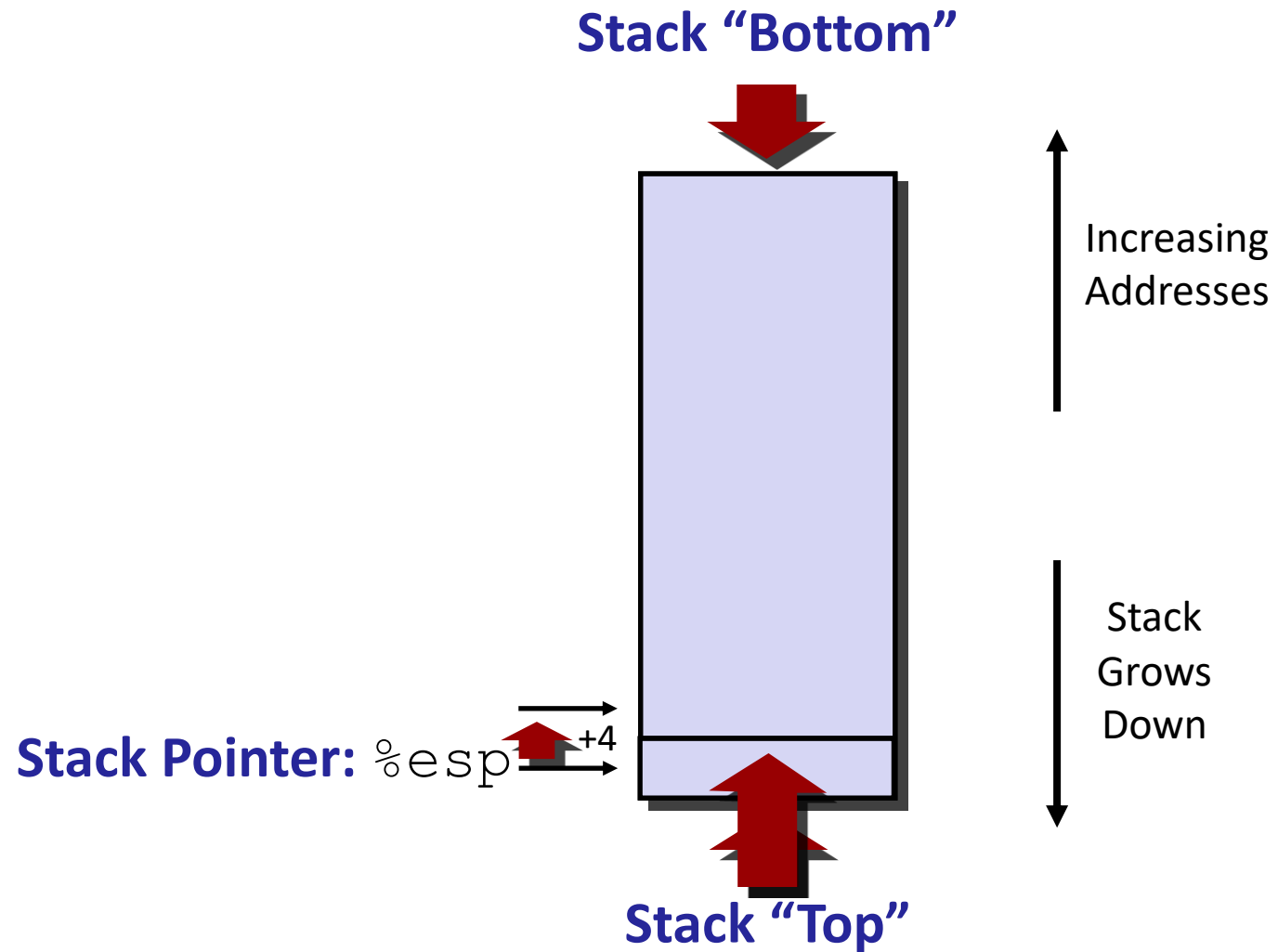
IA32 Stack: Push

■ `pushl Src`

- Fetch operand at *Src*
- Decrement `%esp` by 4
- Write operand at address given by `%esp`



IA32 Stack: Pop



Procedure Control Flow

- Use stack to support procedure call and return

- **Procedure call:** `call label`

- Push return address on stack
- Jump to *label*

- **Return address:**

- Address of the next instruction right after call
- Example from disassembly

```
804854e:  e8 3d 06 00 00    call    8048b90 <main>
8048553:  50               pushl   %eax
```

- Return address = 0x8048553

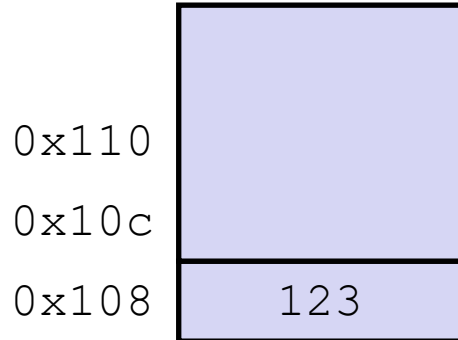
- **Procedure return:** `ret`

- Pop address from stack
- Jump to address

Procedure Call Example

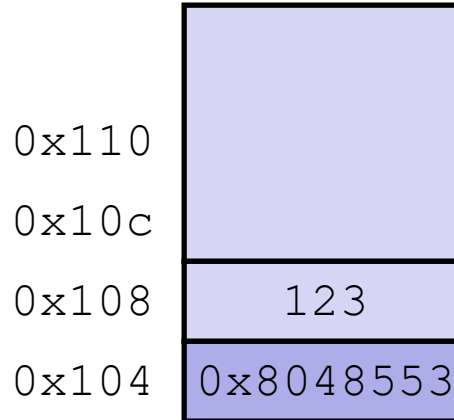
804854e:	e8 3d 06 00 00	call	8048b90 <main>
8048553:	50	pushl	%eax

call 8048b90



%esp 0x108

%eip 0x804854e



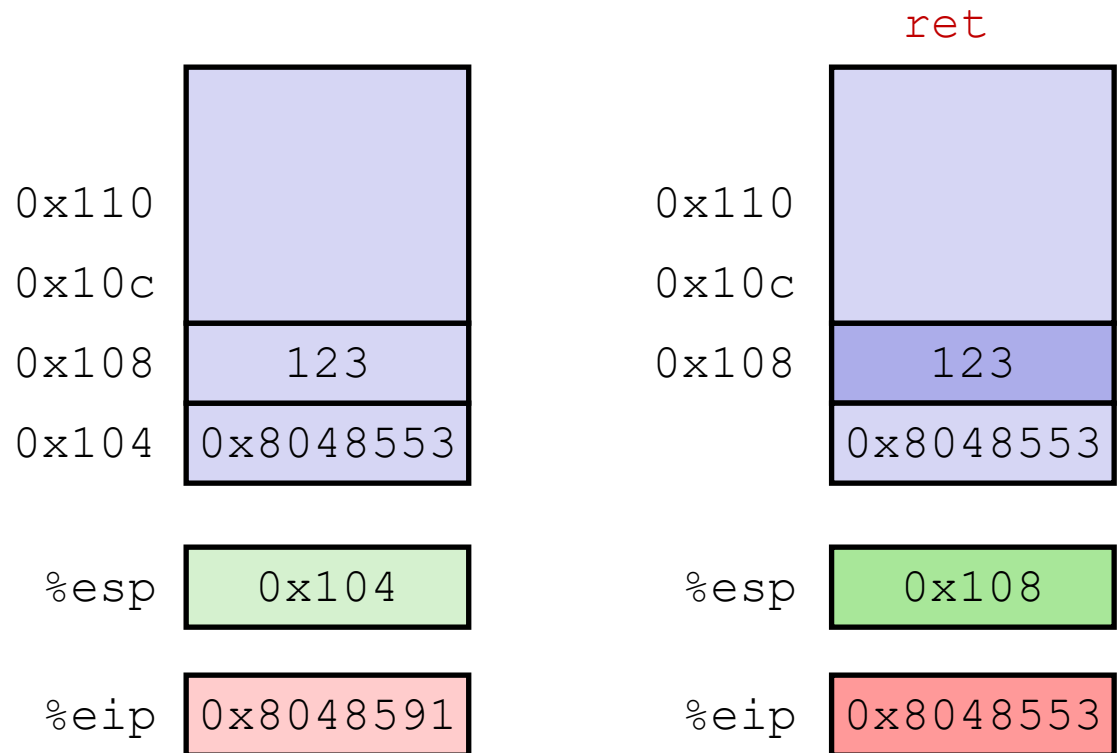
%esp 0x104

%eip 0x8048b90

%eip: *program counter*

Procedure Return Example

8048591: c3 ret



`%eip`: *program counter*

Stack-Based Languages

■ Languages that support recursion

- e.g., C, Pascal, Java
- Code must be “*Reentrant*”
 - Multiple simultaneous instantiations of single procedure
- Need some place to store state of each instantiation
 - Arguments
 - Local variables
 - Return pointer

■ Stack discipline

- State for given procedure needed for limited time
 - From when called to when return
- Callee returns before caller does

■ Stack allocated in *Frames*

- state for single procedure instantiation

Call Chain Example

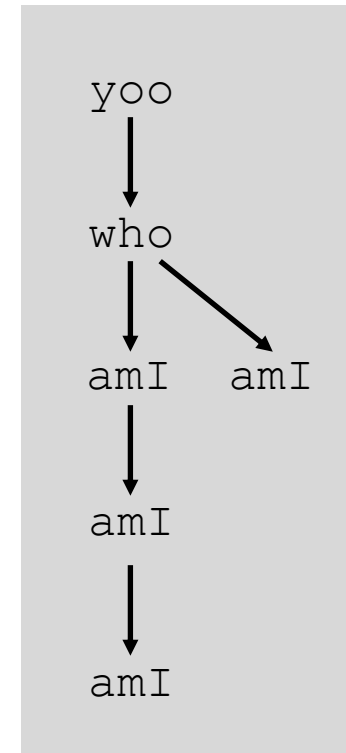
```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```

```
who (...)  
{  
  . . .  
  amI ();  
  . . .  
  amI ();  
  . . .  
}
```

```
amI (...)  
{  
  .  
  .  
  amI ();  
  .  
  .  
}
```

Procedure `amI ()` is recursive

Example Call Chain



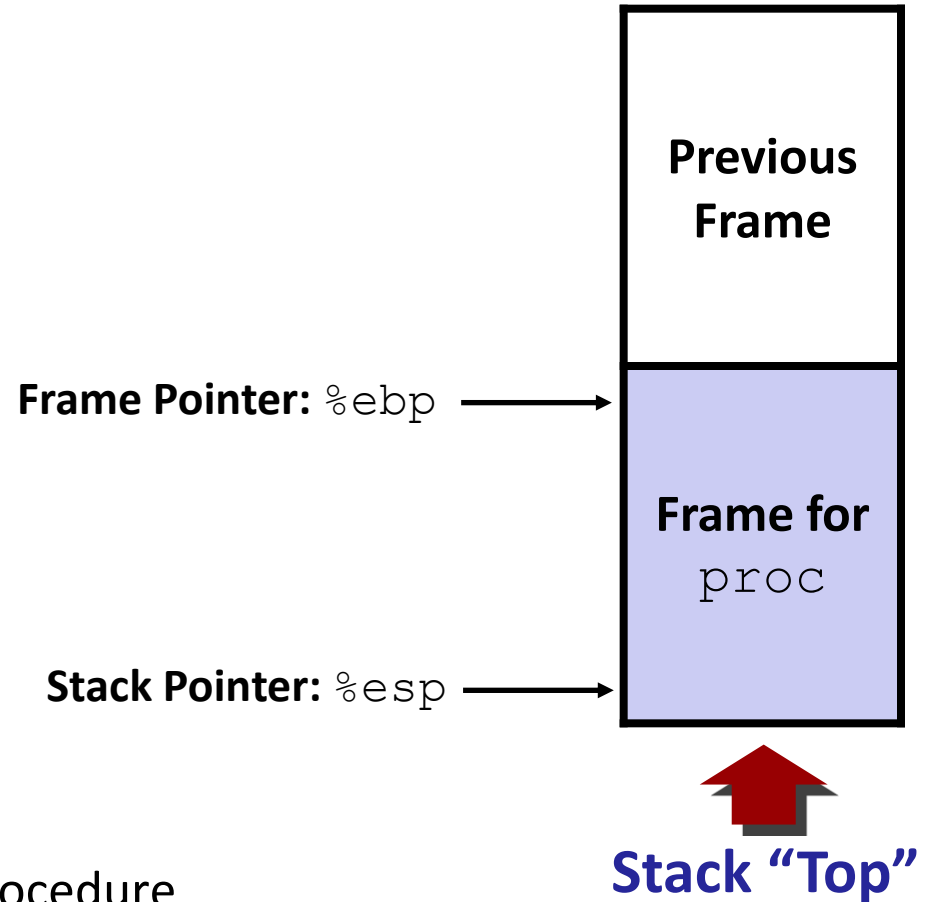
Stack Frames

■ Contents

- Local variables
- Return information
- Temporary space

■ Management

- Space allocated when enter procedure
 - “Set-up” code
- Deallocated when return
 - “Finish” code

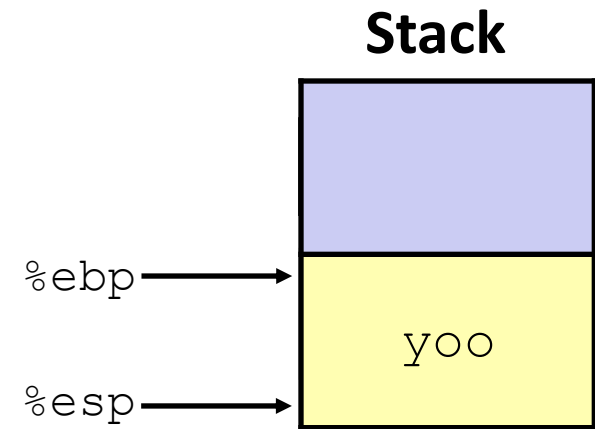


Example

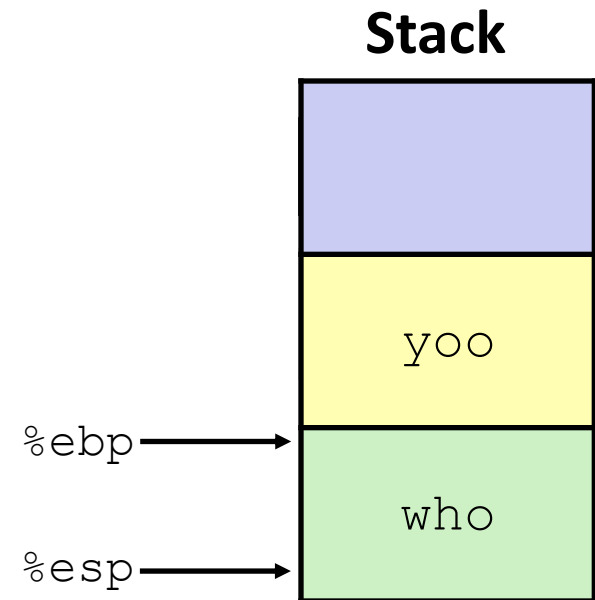
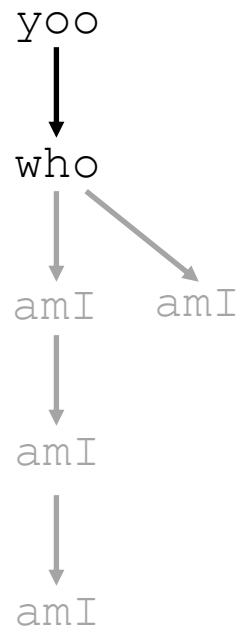
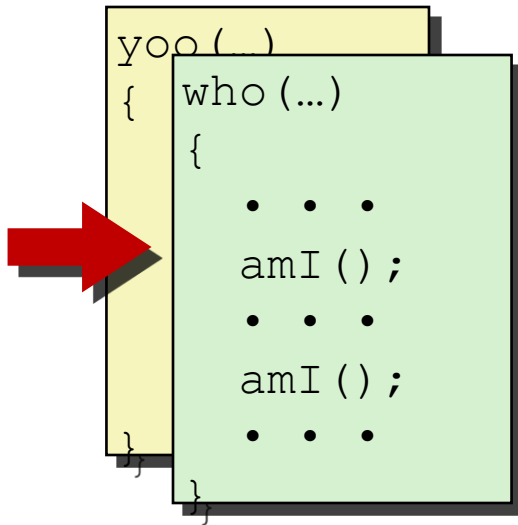


```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```

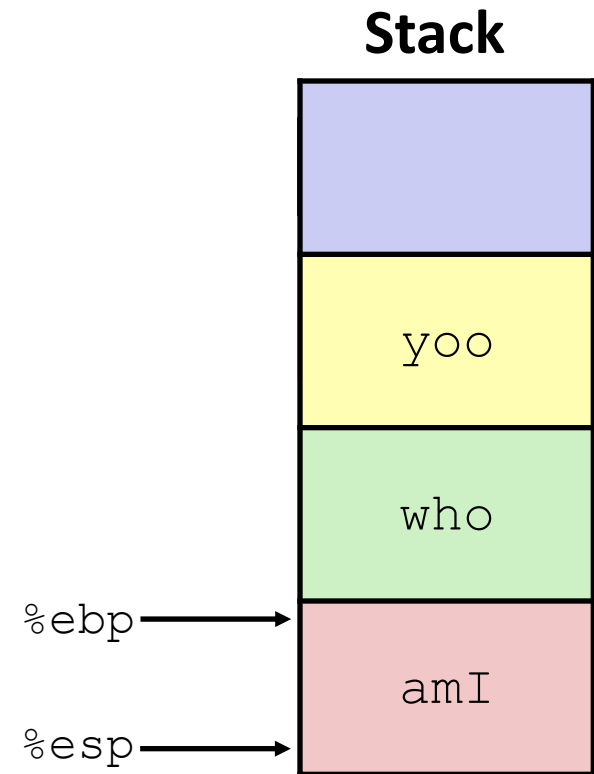
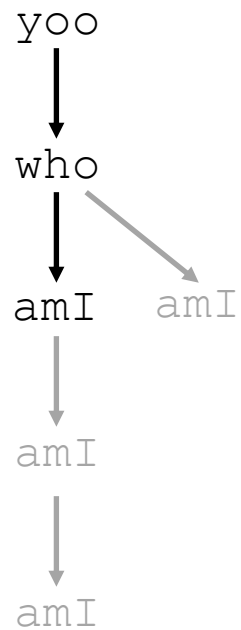
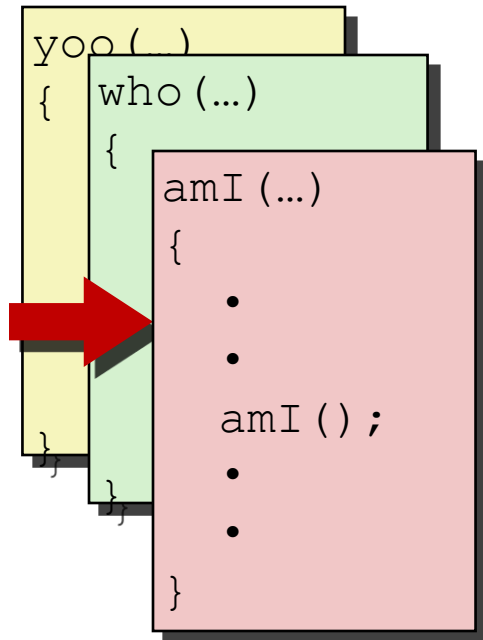
```
yoo  
  ↓  
who  
  ↓  ↘  
amI  amI  
  ↓  
amI  
  ↓  
amI
```



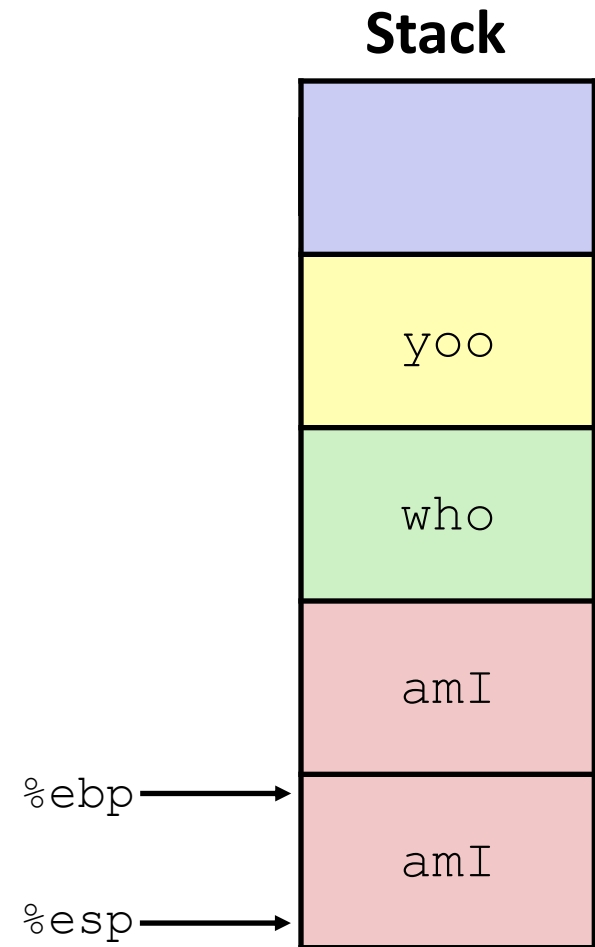
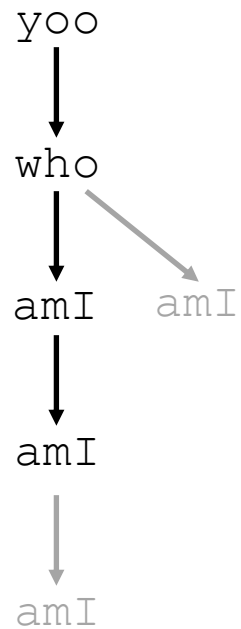
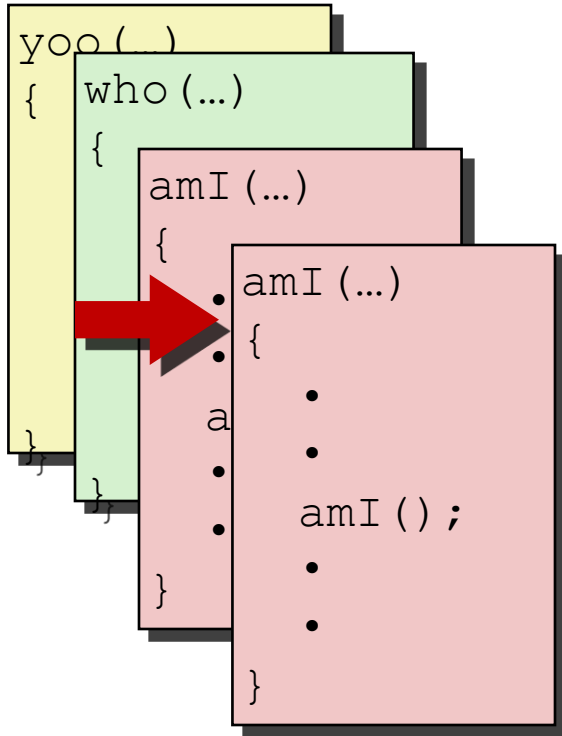
Example



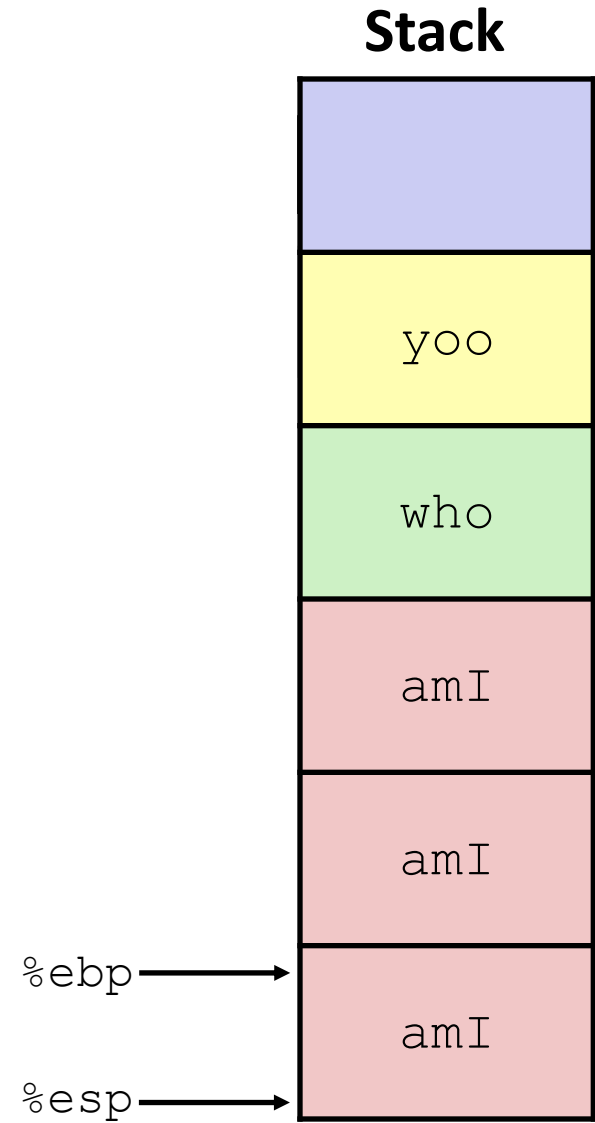
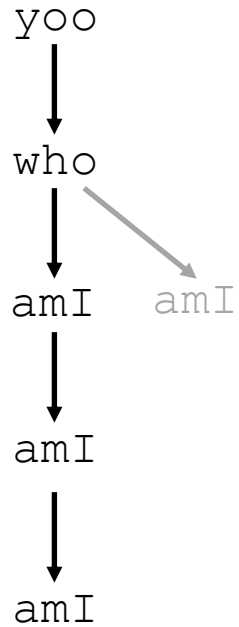
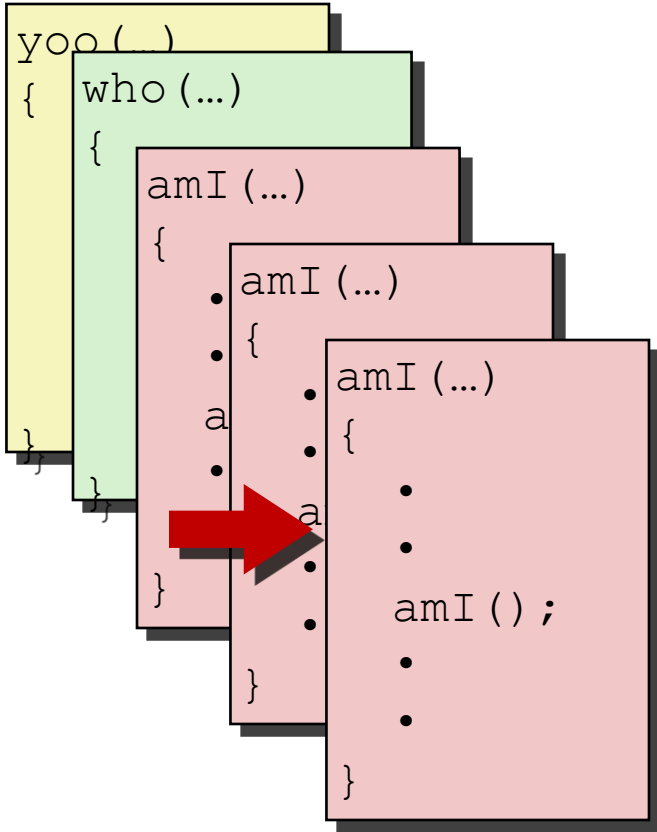
Example



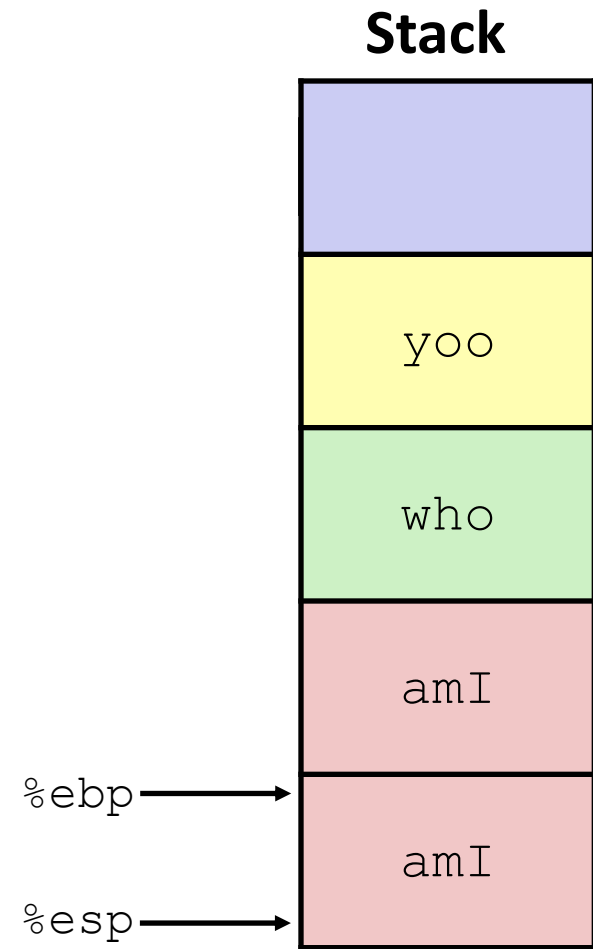
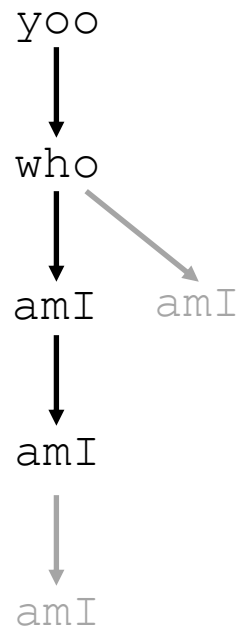
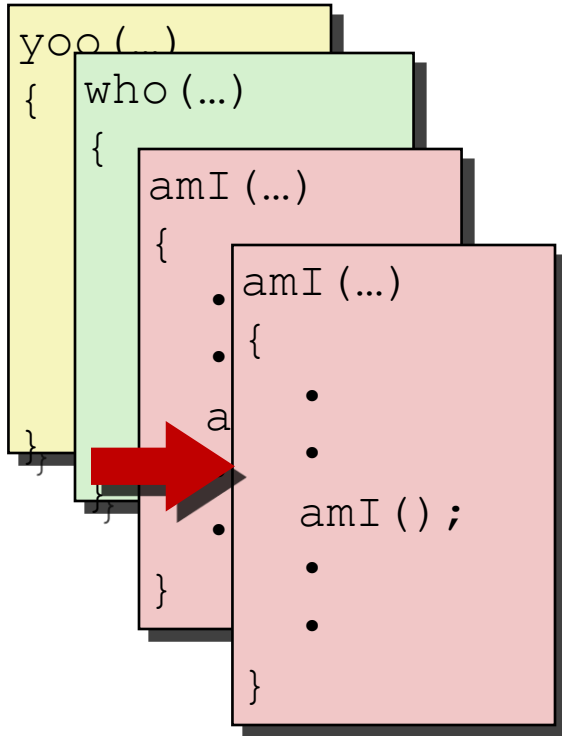
Example



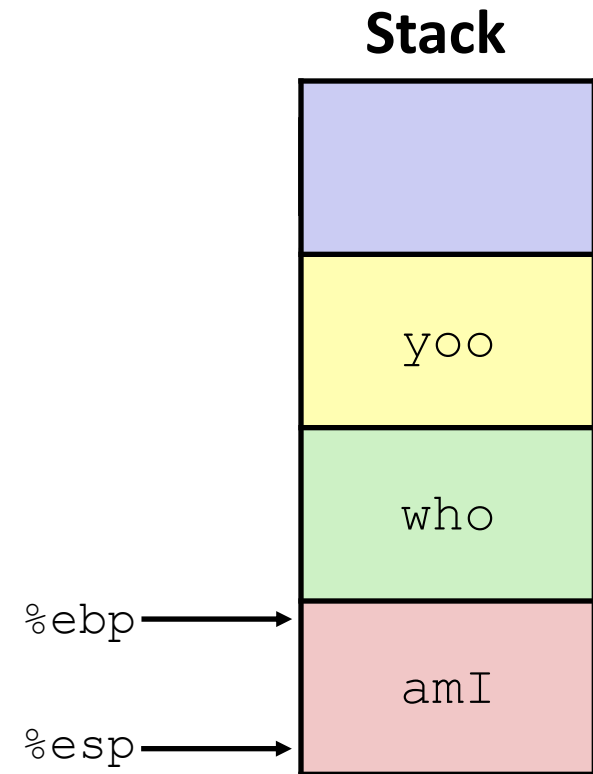
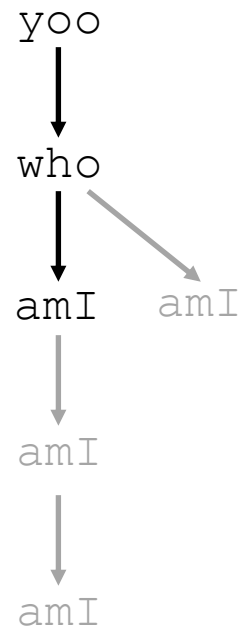
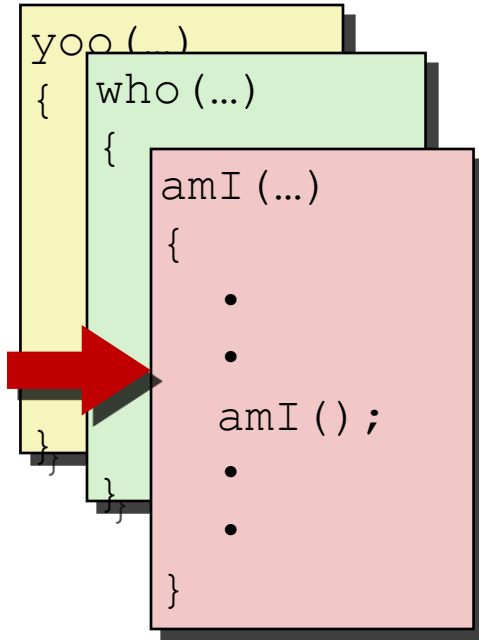
Example



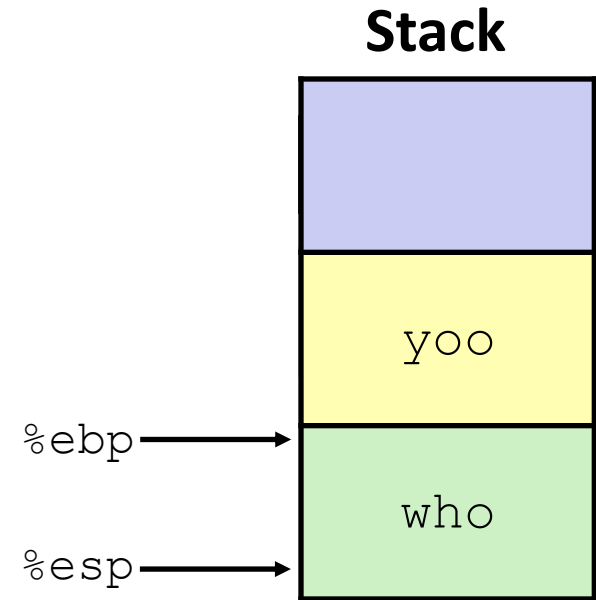
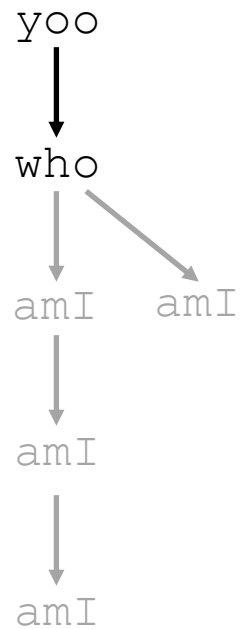
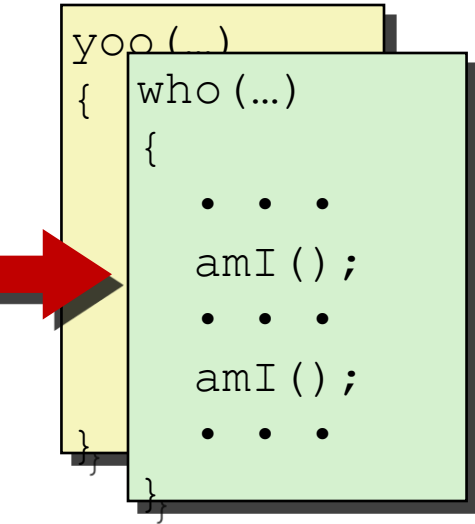
Example



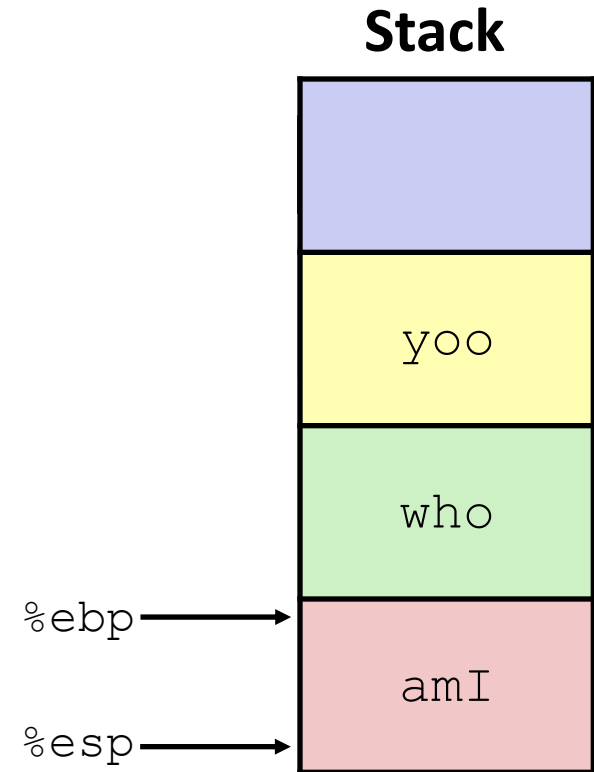
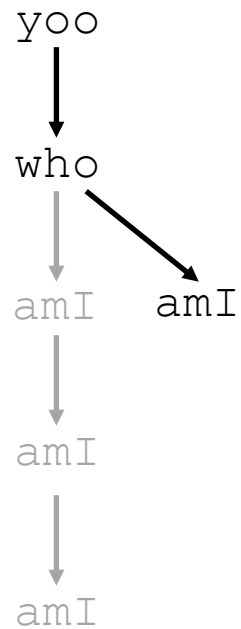
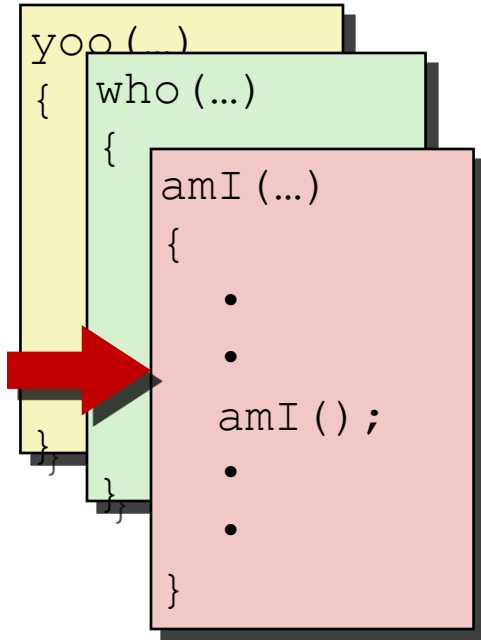
Example



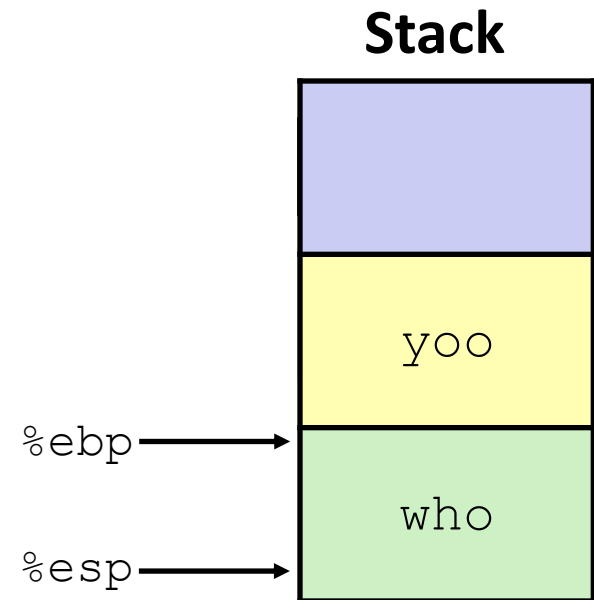
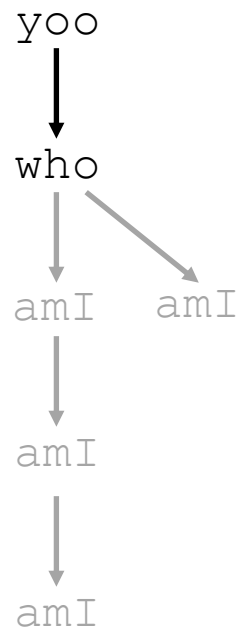
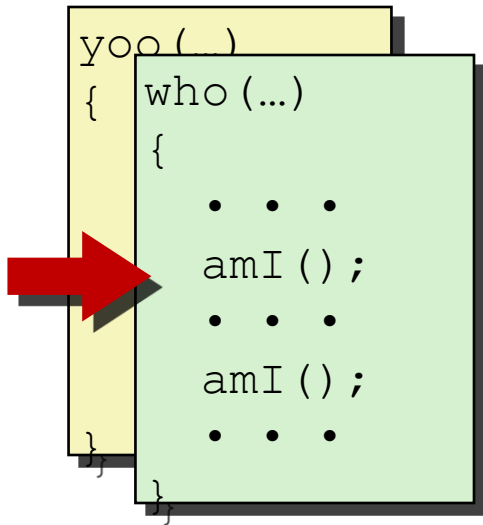
Example



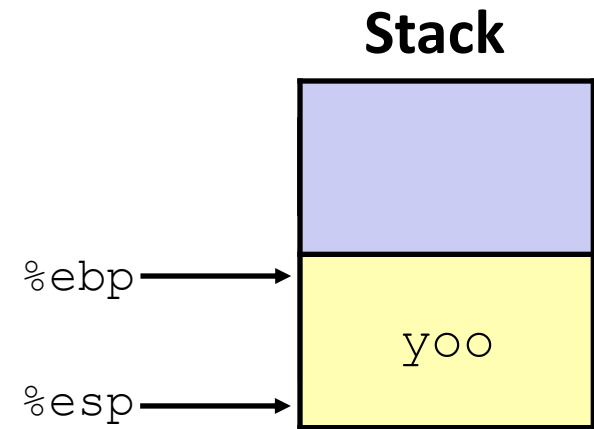
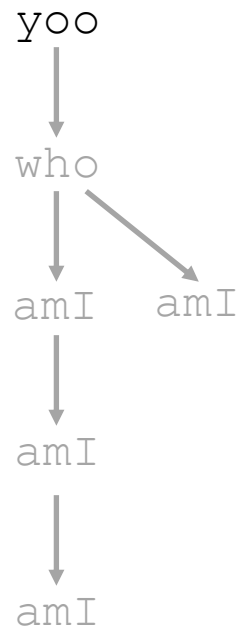
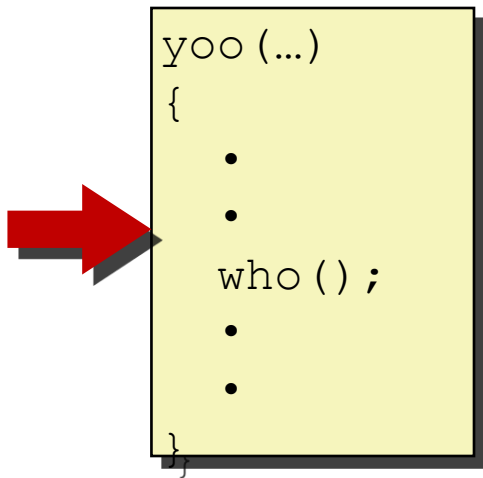
Example



Example



Example



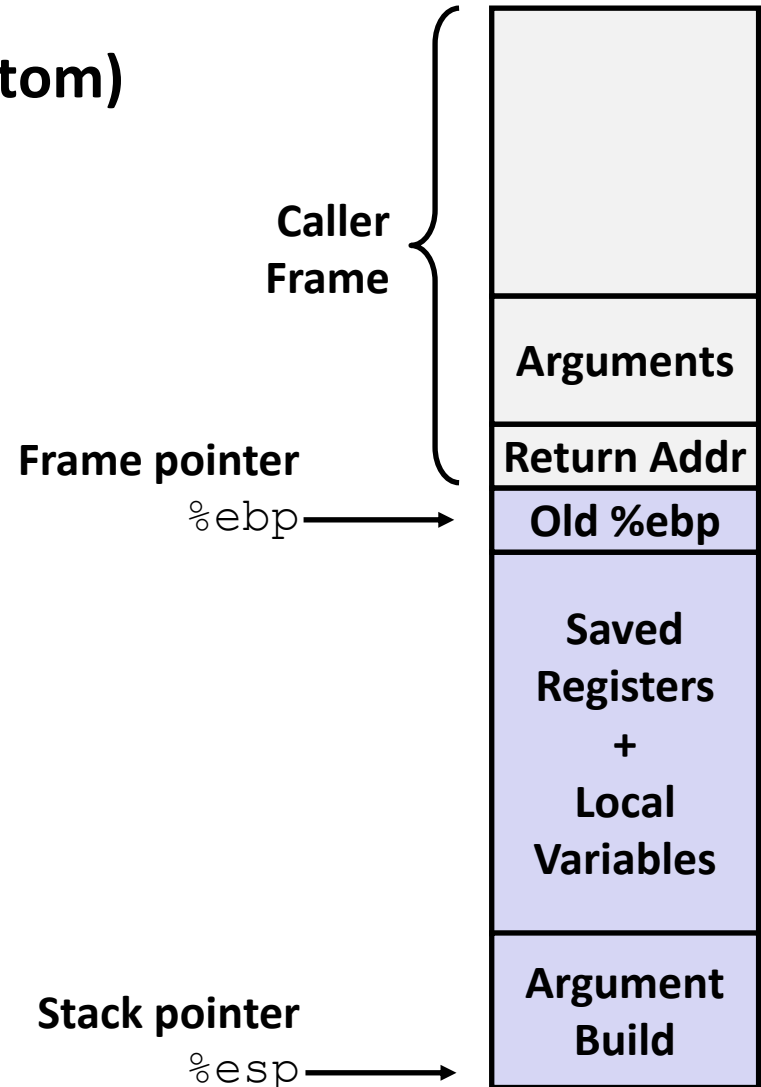
IA32/Linux Stack Frame

■ Current Stack Frame (“Top” to Bottom)

- “Argument build:”
Parameters for function about to call
- Local variables
If can’t keep in registers
- Saved register context
- Old frame pointer

■ Caller Stack Frame

- Return address
 - Pushed by `call` instruction
- Arguments for this call



Revisiting swap

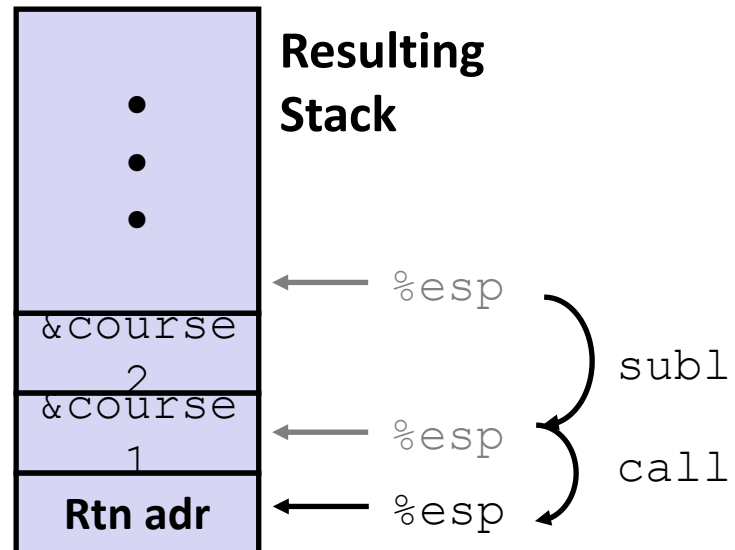
```
int course1 = 15213;
int course2 = 18243;

void call_swap() {
    swap(&course1, &course2);
}
```

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

Calling swap from call_swap

```
call_swap:
    . . .
    subl    $8, %esp
    movl    $course2, 4(%esp)
    movl    $course1, (%esp)
    call    swap
    . . .
```



Recursive Function

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

■ Registers

- `%eax, %edx` used without first saving
- `%ebx` used, but saved at beginning & restored at end

```
pcount_r:
    pushl %ebp
    movl %esp, %ebp
    pushl %ebx
    subl $4, %esp
    movl 8(%ebp), %ebx
    movl $0, %eax
    testl %ebx, %ebx
    je .L3
    movl %ebx, %eax
    shrl %eax
    movl %eax, (%esp)
    call pcount_r
    movl %ebx, %edx
    andl $1, %edx
    leal (%edx, %eax), %eax
.L3:
    addl $4, %esp
    popl %ebx
    popl %ebp
    ret
```


Recursive Call #1

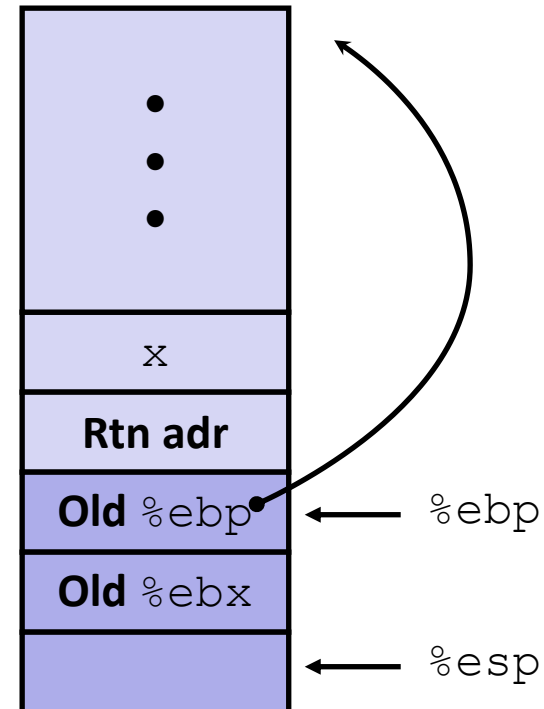
```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

■ Actions

- Save old value of `%ebx` on stack
- Allocate space for argument to recursive call
- Store `x` in `%ebx`



```
pcount_r:
    pushl %ebp
    movl %esp, %ebp
    pushl %ebx
    subl $4, %esp
    movl 8(%ebp), %ebx
    . . .
```



Recursive Call #2

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

```
    . . .
movl  $0, %eax
testl %ebx, %ebx
je    .L3
    . . .
.L3:
    . . .
ret
```

■ Actions

- If `x == 0`, return
 - with `%eax` set to 0

`%ebx`

`x`

Recursive Call #3

```
/* Recursive popcount */  
int pcount_r(unsigned x) {  
    if (x == 0)  
        return 0;  
    else return  
        (x & 1) + pcount_r(x >> 1);  
}
```

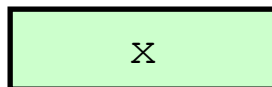
■ Actions

- Store $x \gg 1$ on stack
- Make recursive call

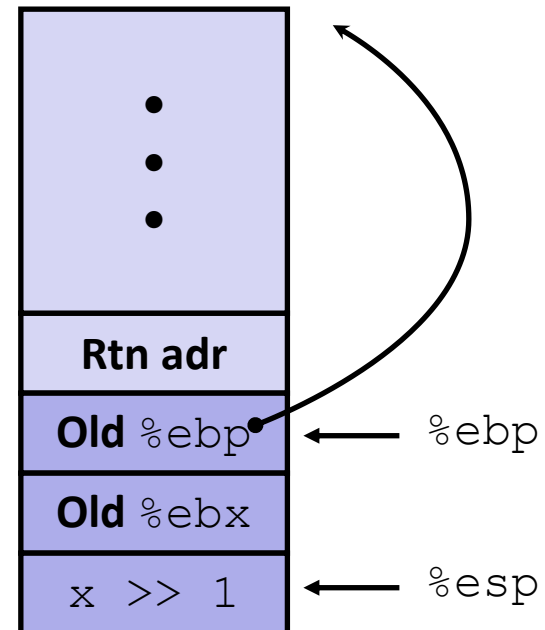
■ Effect

- $\%eax$ set to function result
- $\%ebx$ still has value of x

$\%ebx$



```
• • •  
movl    %ebx, %eax  
shrl    %eax  
movl    %eax, (%esp)  
call    pcount_r  
• • •
```



Recursive Call #4

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

```
• • •
movl    %ebx, %edx
andl    $1, %edx
leal    (%edx,%eax), %eax
• • •
```

■ Assume

- `%eax` holds value from recursive call
- `%ebx` holds `x`



■ Actions

- Compute `(x & 1) + computed value`

■ Effect

- `%eax` set to function result

Recursive Call #5

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

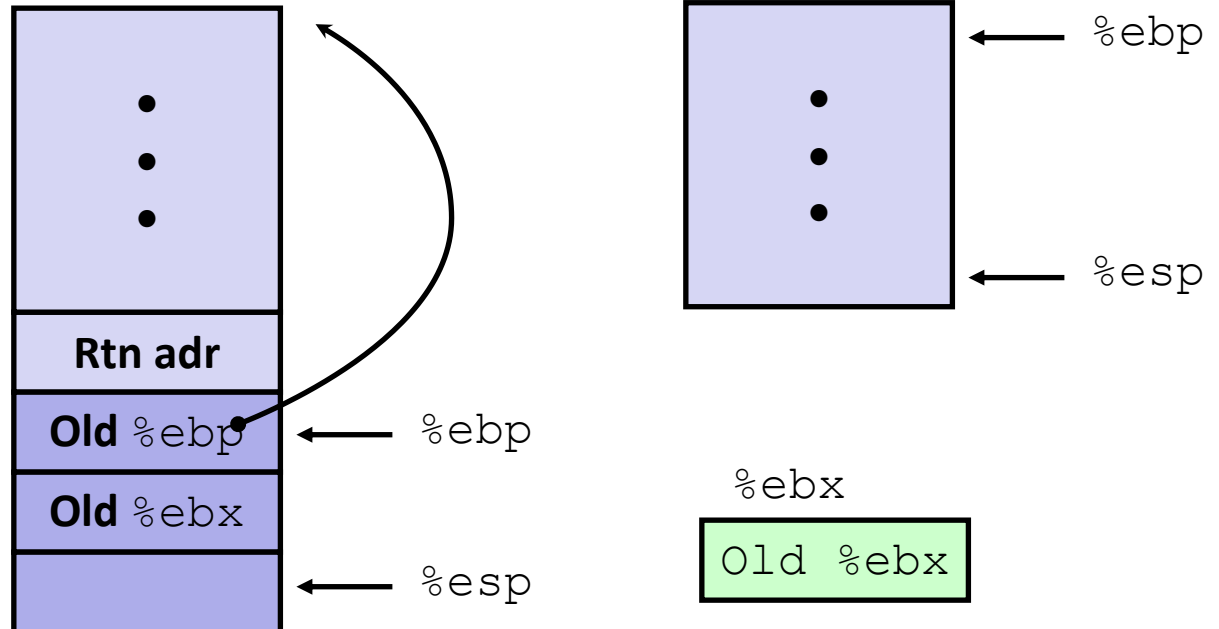
...

L3:

```
addl$4, %esp
popl%ebx
popl%ebp
ret
```

■ Actions

- Restore values of %ebx and %ebp
- Restore %esp



Observations About Recursion

■ Handled Without Special Consideration

- Stack frames mean that each function call has private storage
 - Saved registers & local variables
 - Saved return pointer
- Register saving conventions prevent one function call from corrupting another's data
- Stack discipline follows call / return pattern
 - If P calls Q, then Q returns before P
 - Last-In, First-Out

■ Also works for mutual recursion

- P calls Q; Q calls P